OS ORANGE PROJECT

SRN:PES2UGA24AM154
NAME:SHREYA RAGHURAJ

Screenshot 1A:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$
```

Screenshot 2A:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$
```

Screenshot 2B:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ find .pes/objects -type f
.pes/objects/32/384adec4644a54803d997bd6c8c35bbb21c07f73598d9224a5dfd4e1e1449a
.pes/objects/5c/15fb8dc04503f6c171d9d313a2dbf2848c052faf24088bf0795e689245ab7d
.pes/objects/96/44f3a6870438c8bce0b3d997fb2f77fb780763be734e69b88f635596503b49
```

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ xxd .pes/objects/cb/dda85def8f81f6791334620a4dce293804f956642330cdd24b2829686e65e8
00000000: 7472 6565 2033 3000 7472 6565 2033 3000  tree 30.tree 30.
00000010: 6669 6c65 312e 7478 7400 6861 7368 3132  file1.txt.hash12
00000020: 3334 3536 3738                           345678
```

Screenshot 3A:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ ./pes add file1.txt file2.txt
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ ./pes status
Staged changes:
  staged:     phase3_test.txt
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.c
  untracked:  commit.h
  untracked:  hello.txt
  untracked:  object.c
  untracked:  .gitignore
  untracked:  test_objects.c
  untracked:  object.h
  untracked:  snapshot_test.txt
  untracked:  README.md
  untracked:  tree.c
  untracked:  commit.c
  untracked:  index.h
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  test_file.txt
  untracked:  pes.c
  untracked:  pes.h
```

Screenshot 3B:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ cat .pes/index
100664 8e821e10154265031d0403a69d63fb20a5356252e0762ca90bf2129d6a9c40dc 1776423343 17 phase3_test.txt
100664 999f24152159e51756a944d32257bf22080ff8608fff87ca9a4a823764e13dbe 1776411119 5 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776410772 6 file2.txt
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$
```

Screenshot 4A:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ ./pes log
commit 397965a22ae65858d104991cf57f97ef901107560fda1edc5008341906554200
Author: SHREYA <PES2UG24AM154>
Date:   1776425500

    Third commit: final update

commit bcf5938ec36a532c708ba3b5062906c9f742f3d72e43fc2e4484bcfffbd2fef7
Author: SHREYA <PES2UG24AM154>
Date:   1776425500

    Second commit: updated file1_cert_id!

commit ae5105c44298424f8e431e6553499ee3da7b399054c169dee5e689998cdc5161
Author: SHREYA <PES2UG24AM154>
Date:   1776425500

    First commit: added file1

commit 442d26bcdfb17b81612d0b5c948875eefaa2bbdc11a446ba7fe0a4e1897cb2e1
Author: SHREYA <PES2UG24AM154>
Date:   1776425382

    Final submission
```

Screenshot 4B:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/02/3f4a9f53ea59717642db725b93a61c645b1b560acd92636d64461bb08ee62a
.pes/objects/39/7965a22ae65858d104991cf57f97ef901107560fda1edc5008341906554200
.pes/objects/41/0370663e9874f8c898a936157f90602b3a863f54f25befacb3add87128b2a8
.pes/objects/44/2d26bcdfb17b81612d0b5c948875eefaa2bbdc11a446ba7fe0a4e1897cb2e1
.pes/objects/49/31da8dd77e4a471b71045e681518be0e30380ee3414e146ca45f03daf7d6b4
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/5b/395e18222f42cd25372ff565689333f32a871608658c0e597c1deb6c5244a7
.pes/objects/77/c72fdc2899a01315d2c4b0a0396d2c72077752c3b9d88d736d910917cd7f23
.pes/objects/ae/5105c44298424f8e431e6553499ee3da7b399054c169dee5e689998cdc5161
.pes/objects/bc/f5938ec36a532c708ba3b5062906c9f742f3d72e43fc2e4484bcfffbd2fef7
.pes/objects/bd/965fae20eb0dd136c5fb90b7c99ea1863ab46bb9a2647be049f38751749e61
.pes/objects/d5/f0969caa025c0921a2822efd9f9b316712a6467d04ff391b3fe0ac61881d8c
.pes/refs/heads/main
```

Screenshot 4C:

```
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ cat .pes/refs/heads/main
397965a22ae65858d104991cf57f97ef901107560fda1edc5008341906554200
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
shreya@ubuntu:~/PES2UG24AM154-pes-vcs$
```

PHASE 5 & 6:

Q.5.1 To implement `pes checkout`, the following changes must occur within the `.pes/` directory and the working directory:

- **`.pes/` Changes**: The `HEAD` file must be updated to contain the reference string for the target branch (e.g., `ref: refs/heads/<branch>`).
- **Working Directory Changes**: Every file in the current directory must be removed or overwritten to match the state of the tree object associated with the target branch's latest commit.
- **Complexity**: This operation is complex because it is destructive. If the working directory has untracked files, the system must decide whether to keep them or delete them. Furthermore, mapping a recursive tree object back into a flat physical directory structure requires careful file I/O and permission handling.

Q.5.2.

To detect a conflict before checking out a branch, the system compares the current working directory against the index and the object store:

1. **Compare Working Directory to Index**: For every tracked file, calculate the current hash of the file on disk. If the hash differs from the hash stored in the `.pes/index`, the file is "dirty."
2. **Compare Index to Target Branch**: Compare the hashes in the current index with the hashes in the tree of the target branch commit.
3. **Conflict Detection**: If a file is "dirty" (differs from the current index) **and** that file's version in the target branch is different from the version in the current index, the checkout must be refused to prevent the user's uncommitted work from being overwritten.

Q.5.3

In a "Detached HEAD" state, `HEAD` points directly to a commit hash rather than a branch name.

- **What happens**: Commits function normally, but each new commit's `parent` is the previous hash in `HEAD`. However, because no branch name (like `main`) is being updated, these new commits are not part of any branch.
- **Recovery**: If the user switches away to another branch, the hashes of the "detached" commits are lost from `HEAD`. To recover them, a user must find the hash of the last detached commit (using a tool like `reflog` or terminal history) and manually create a new branch pointing to that hash: `echo <hash> > .pes/refs/heads/new-branch-name`.

Q.6.1

An algorithm to clean the object store is known as **Mark-and-Sweep**:

- **Algorithm**:
    1. **Mark**: Start at all "references" (files in `refs/heads/` and `HEAD`). Recursively follow the hashes: Commit → Tree → Sub-trees/Blobs. Every hash encountered is added to a "reachable" set.
    2. **Sweep**: Iterate through every file in `.pes/objects/`. If a file's hash is not in the "reachable" set, delete it.
- **Data Structure**: A **Bloom Filter** or a **Hash Set** is best for tracking reachable hashes efficiently as it allows for O(1) lookup time.
- **Estimation**: For 50 branches and 100,000 commits, you would visit at least **100,000 commit objects**. Assuming each commit averages 1 root tree and 2-3 modified blobs/sub-trees, you would likely visit **300,000 to 500,000 objects** total.

Q.6.2.Running GC during a commit is dangerous because of the time gap between an object being written and being referenced.

- **Race Condition**:
    1. A `pes add` operation creates a new blob and writes it to the object store.
    2. The GC starts, checks all branches, and sees that no commit points to this new blob yet (because the `commit`command hasn't finished).
    3. The GC deletes the "unreferenced" blob.
    4. The `commit` command finishes and tries to link to that blob, resulting in a corrupted repository.
- **Avoidance**: Real Git GC avoids this by using a **grace period** (the `prune` timer). It will only delete unreachable objects that are older than a specific threshold (typically 2 weeks), ensuring that any "in-progress" objects are safe from deletion.